

Agent Harness Scaffold

Agent Harness Scaffold (Node.js / Python, Redis-Backed)

Overview

This document provides a complete **Agent Harness scaffold** for your event-driven, Redis-backed architecture. The Agent Harness is the service that:

- Subscribes to Redis task queues
- Runs agents (LLM, tools, workflows)
- Publishes logs and events back to Redis
- Supports human-in-the-loop pause/resume
- Restores agent state from snapshots

You can implement this in **Node.js**, **Python**, or both. This scaffold focuses on Node.js first, with a Python outline at the end.

Folder Structure (Node.js Agent Harness)

```
/agent-harness-node
worker.ts
/redis
sub.ts
pub.ts
/runtime
mockAgentRuntime.ts
stateMachine.ts
snapshotStore.ts
/events
eventTypes.ts
emitEvents.ts
/db
TaskModel.ts
```

File: /redis/pub.ts

```
import Redis from "ioredis";

export const redisPub = new Redis({
  host: "localhost",
  port: 6379
});

export async function publishEvent(channel: string, payload: any) {
  await redisPub.publish(channel, JSON.stringify(payload));
}
```

File: /redis/sub.ts

```
import Redis from "ioredis";
import { handleTaskMessage } from "../runtime/stateMachine";

export const redisSub = new Redis({
  host: "localhost",
  port: 6379
});

export async function startTaskSubscriber() {
  await redisSub.subscribe("tasks");

  redisSub.on("message", async (channel, message) => {
    if (channel !== "tasks") return;
    const payload = JSON.parse(message);
    await handleTaskMessage(payload);
  });
}
```

File: /events/eventTypes.ts

```
export type AgentEventType =
  | "log.append"
  | "output.partial"
  | "output.final"
  | "session.stateChanged"
  | "approval.requested";
```

 File: /events/emitEvents.ts

```

import { publishEvent } from "../redis/pub";
import { AgentEventType } from "../eventTypes";

export async function emitLog(sessionId: string, message: string, severity: string = "info") {
  await publishEvent("agent.logs", {
    sessionId,
    type: "log.append" as AgentEventType,
    payload: { message, severity }
  });
}

export async function emitPartialOutput(sessionId: string, chunk: string) {
  await publishEvent(`session:${sessionId}:events`, {
    sessionId,
    type: "output.partial" as AgentEventType,
    payload: { chunk }
  });
}

export async function emitFinalOutput(sessionId: string, output: any) {
  await publishEvent(`session:${sessionId}:events`, {
    sessionId,
    type: "output.final" as AgentEventType,
    payload: { output }
  });
}

export async function emitSessionState(sessionId: string, to: string) {
  await publishEvent(`session:${sessionId}:events`, {
    sessionId,
    type: "session.stateChanged" as AgentEventType,
    payload: { to }
  });
}

export async function emitApprovalRequest(taskId: string, sessionId: string, promptMessage:
string, options: string[]) {
  await publishEvent("agent.events", {
    sessionId,
    type: "approval.requested" as AgentEventType,
    payload: { taskId, promptMessage, options }
  });
}

```

File: /db/TaskModel.ts

```
// Placeholder DB model. Replace with real MongoDB/Postgres implementation.
```

```
export const TaskModel = {
  async findById(taskId: string) {
    // TODO: Load from DB
    return {
      id: taskId,
      status: "PENDING",
      agentId: "mock-agent",
      input: {},
      humanInteraction: null,
      agentInternalSnapshot: null
    };
  },

  async update(taskId: string, data: any) {
    // TODO: Update DB
    return true;
  }
};
```

File: /runtime/snapshotStore.ts

```
import { TaskModel } from "../db/TaskModel";

export async function loadSnapshot(taskId: string) {
  const task = await TaskModel.findById(taskId);
  return task.agentInternalSnapshot;
}

export async function saveSnapshot(taskId: string, snapshot: any) {
  await TaskModel.update(taskId, { agentInternalSnapshot: snapshot });
}
```

File: /runtime/mockAgentRuntime.ts

```
import { emitLog, emitPartialOutput, emitFinalOutput, emitSessionState,
emitApprovalRequest } from "../events/emitEvents";
import { saveSnapshot } from "./snapshotStore";

export async function runMockAgent(taskId: string, sessionId: string, input: any) {
  await emitSessionState(sessionId, "running");
  await emitLog(sessionId, "Mock agent started", "info");

  // Simulate some work
  await emitPartialOutput(sessionId, "Working on task...\n");

  // Simulate human-in-the-loop breakpoint
  await saveSnapshot(taskId, {
    step: "awaiting_choice",
    input
  });

  await emitApprovalRequest(
    taskId,
    sessionId,
    "Select optimization mode",
    ["Standard", "Performance Optimized"]
  );

  // At this point, the agent should pause and let the Gateway + user respond.
}

export async function resumeMockAgent(taskId: string, sessionId: string, choice: string) {
  await emitLog(sessionId, `Resuming with choice: ${choice}`, "info");

  // Simulate resumed work
  await emitPartialOutput(sessionId, `Applying choice: ${choice}...\n`);

  await emitFinalOutput(sessionId, {
    result: "Task completed",
    choice
  });

  await emitSessionState(sessionId, "completed");
}
```

File: /runtime/stateMachine.ts

```
import { TaskModel } from "../db/TaskModel";
import { runMockAgent, resumeMockAgent } from "../mockAgentRuntime";

export async function handleTaskMessage(message: any) {
  switch (message.type) {
    case "task.created": {
      const task = await TaskModel.findById(message.taskId);
      const sessionId = message.taskId; // or a separate session mapping
      await runMockAgent(task.id, sessionId, task.input);
      break;
    }

    case "task.resume": {
      const task = await TaskModel.findById(message.taskId);
      const sessionId = message.taskId;
      const choice = task.humanInteraction?.selectedChoice || "Standard";
      await resumeMockAgent(task.id, sessionId, choice);
      break;
    }

    default:
      // Ignore unknown messages
      break;
  }
}
```

File: worker.ts

```
import { startTaskSubscriber } from "../redis/sub";

async function main() {
  console.log("Agent Harness (Node) starting...");
  await startTaskSubscriber();
}

main().catch(err => {
  console.error("Agent Harness failed", err);
  process.exit(1);
});
```

Python Agent Harness Outline

For Python, the structure mirrors the Node harness:

```
/agent-harness-python
worker.py
redis_sub.py
redis_pub.py
runtime/
  mock_agent_runtime.py
  state_machine.py
  snapshot_store.py
db/
  task_model.py
events/
  emit_events.py
```

Example: worker.py

```
from redis_sub import start_task_subscriber

if __name__ == "__main__":
    print("Agent Harness (Python) starting...")
    start_task_subscriber()
```

End-to-End Flow

With this Agent Harness scaffold, your full pipeline becomes:

```
Frontend (HTTP + SSE)
  ↓ ↑
Gateway (Fastify)
  ↓ ↑
Redis (tasks, agent.logs, agent.events, session:{id}:events)
  ↓ ↑
Agent Harness (Node/Python)
```

- Gateway publishes task.created and task.resume to tasks.
- Agent Harness consumes tasks and runs agents.
- Agent Harness publishes logs and outputs to agent.logs, agent.events, and session:{id}:events.
- Gateway exposes SSE on /sessions/:sessionId/events.
- Frontend SSE client consumes events and routes them into the RxJS bus.

Summary

This Agent Harness scaffold gives you:

- Redis subscriber for task messages
- Redis publisher for logs and session events
- Mock agent runtime with human-in-the-loop pause/resume
- Snapshot storage hooks
- A clear state machine for task.created and task.resume

It is fully aligned with your Gateway, frontend SSE client, and workflow engine scaffolds, and ready to be implemented in Node.js and/or Python.